

School of Computer Science and Artificial Intelligence

Lab Assignment # 8.5

Program	: B. Tech (CSE)
Specialization	: CSE
Course Title	: AI Assisted coding
Course Code	: 23CS002PC304
Semester	: II
Academic Session	: 2025-2026
Name of Student	: A VamshiKrishna
Enrollment No.	: 2403A51L47
Batch No.	: 52
Date	: 30-01-2026

Task 1 :

Use AI to generate at least 3 assert test cases for a function `is_valid_username(username)` and then implement the function using Test-Driven Development principles.

Prompt:

Generate at least 3 assert test cases for a Python function `is_valid_username(username)` using Test-Driven Development (TDD). Then implement the function so that all test cases pass.

Requirements:

- Username length must be between 5 and 15 characters.
- Must contain only alphabets and digits.
- Must not start with a digit.
- No spaces allowed.
- Provide the full Python code including the test cases and function implementation.

```
def is_valid_username(username):  
    if len(username) < 5 or len(username) > 15:  
        return False  
  
    if username[0].isdigit():  
        return False  
  
    if not username.isalnum():  
        return False  
  
    return True  
  
def run_tests():  
    assert is_valid_username("User123") == True  
    assert is_valid_username("12User") == False  
    assert is_valid_username("Us er") == False  
    assert is_valid_username("Usr1") == False  
    assert is_valid_username("VeryLongUsername123") == False  
    assert is_valid_username("User_123") == False  
    assert is_valid_username("Username") == True  
    assert is_valid_username("Alpha123") == True  
  
    print("Username validation logic successfully passing all AI-generated test cases")  
  
run_tests()
```

```
C:\Users\atur\OneDrive\Desktop\Ai_assistant>C:/Python314/python.exe c:/Users/atur/OneDrive/Desktop/Ai_as
Username validation logic successfully passing all AI-generated test cases
```

Explanations:

- The function checks the length, ensures the username does not start with a digit, and verifies that it contains only letters and numbers using `isalnum()`.
- Assert tests are written first following TDD principles, ensuring the function works correctly before confirming success.

Task 2 : AI to generate at least 3 assert test cases for a function `classify_value(x)` and implement it using conditional logic and loops.

Prompt:

Generate at least 3 assert test cases for a Python function `classify_value(x)` and then implement the function using conditional statements and loops.

Requirements:

- If the input is an integer, return "Even" or "Odd".
- If the input is 0, return "Zero".
- If the input is non-numeric, return "Invalid Input".
- Provide the full Python code including the assert test cases and the function implementation that passes all tests.

```
def classify_value(x):
    if isinstance(x, int):
        if x == 0:
            return "Zero"
        elif x % 2 == 0:
            return "Even"
        else:
            return "Odd"

    if isinstance(x, str):
        if len(x) == 0:
            return "Invalid Input"

        for ch in x:
            if ch not in "0123456789":
                return "Invalid Input"

        num = int(x)

        if num == 0:
            return "Zero"
        elif num % 2 == 0:
            return "Even"
        else:
            return "Odd"

    return "Invalid Input"

def run_tests():
    assert classify_value(8) == "Even"
    assert classify_value(7) == "Odd"
    assert classify_value("abc") == "Invalid Input"
    assert classify_value(0) == "Zero"
    assert classify_value("12") == "Even"
    assert classify_value("9") == "Odd"
    assert classify_value("") == "Invalid Input"
    print("Function correctly classifying values and passing all test cases")

run_tests()
```

```
C:\Users\atur\OneDrive\Desktop\Ai_assistanr>python -u "c:\Users\atur\OneDrive\Desktop\Ai_assistanr\palindrome_checker.py"
Function correctly classifying values and passing all test cases
```

Explanation :

- This prompt asks AI to first create test cases and then implement the function following TDD principles.
- The function classifies inputs based on whether they are Even, Odd, Zero, or Invalid.
- All generated test cases should pass successfully.

Task 3 (Palindrome Checker – Apply AI for String Normalization):

Use AI to generate at least 3 assert test cases for a function `is_palindrome(text)` and implement the function.

Prompt :

Generate at least 3 assert test cases for a Python function `is_palindrome(text)` and then implement the function.

Requirements:

- Ignore case differences.
 - Ignore spaces and punctuation.
 - Handle edge cases such as empty strings and single characters.
- Provide the full Python code including the assert test cases and the function implementation that passes all tests.

```
def is_palindrome(text):
    normalized = ""
    for ch in text:
        if ch.isalnum():
            normalized += ch.lower()
    return normalized == normalized[::-1]

def run_tests():
    assert is_palindrome("Madam") == True
    assert is_palindrome("A man a plan a canal Panama") == True
    assert is_palindrome("Python") == False
    assert is_palindrome("") == True
    assert is_palindrome("a") == True
    assert is_palindrome("No lemon, no melon") == True
    print("Function correctly identifying palindromes and passing all AI-generated tests")

run_tests()
```

```
C:\Users\atur\OneDrive\Desktop\Ai_assistanr>C:/Python314/python.exe c:/Users/atur/OneDrive/Desktop/Ai_assistanr/palindrome_checker.py
Function correctly identifying palindromes and passing all AI-generated tests
```

Explanation:

- This prompt asks AI to create test cases first and then implement the palindrome-checking function.
- The function should normalize the string by removing spaces and punctuation and ignoring case.
- The final code must pass all the generated test cases.

Task 4 (BankAccount Class – Apply AI for Object-Oriented Test-Driven Development):

- Task: Ask AI to generate at least 3 assert-based test cases for a BankAccount class and then implement the class.

Prompt:

Generate at least 3 assert-based test cases for a Python class BankAccount and then implement the class using Object-Oriented Programming and Test-Driven Development principles.

Methods required:

- deposit(amount)
- withdraw(amount)
- get_balance()

The class should correctly update and return the account balance. Provide the full Python code including the assert test cases and class implementation that passes all tests.

```
class BankAccount:
    def __init__(self, balance):
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount

    def get_balance(self):
        return self.balance

def run_tests():
    acc = BankAccount(1000)
    acc.deposit(500)
    assert acc.get_balance() == 1500

    acc.withdraw(300)
    assert acc.get_balance() == 1200

    acc.deposit(200)
    assert acc.get_balance() == 1400

    acc.withdraw(1000)
    assert acc.get_balance() == 400

    print("Fully functional class that passes all AI-generated assertions")
```

```
run_tests()
```

```
C:\Users\atur\OneDrive\Desktop\Ai_assistant>C:/Python314/python.exe c:/Users/atur/OneDrive/
Fully functional class that passes all AI-generated assertions
```

Explanation:

- The BankAccount class manages a user's balance using object-oriented programming.
- The deposit() method adds money, withdraw() removes money if enough balance is available, and get_balance() returns the current balance.
- Assert test cases verify that the class works correctly following Test-Driven Development (TDD)

Task 5 (Email ID Validation – Apply AI for Data Validation):

Use AI to generate at least 3 assert test cases for a function validate_email(email) and implement the function.

Prompt

Generate at least 3 assert test cases for a Python function validate_email(email) and then implement the function.

Requirements:

- Email must contain @ and .
 - Must not start or end with special characters.
 - Should handle invalid formats gracefully.
- Provide the full Python code including the assert test cases and the function implementation that passes all tests.

```
def validate_email(email):
    if not isinstance(email, str):
        return False

    if len(email) == 0:
        return False

    if email[0] in "@._" or email[-1] in "@._":
        return False

    if "@" not in email or "." not in email:
        return False

    at_count = 0
    for ch in email:
        if ch == "@":
            at_count += 1

    if at_count != 1:
        return False

    return True

def run_tests():
    assert validate_email("user@example.com") == True
    assert validate_email("userexample.com") == False
    assert validate_email("@gmail.com") == False
    assert validate_email("user@domain") == False
    assert validate_email("user.name@mail.com") == True
    assert validate_email("user@mail.") == False
    print("Email validation function passing all AI-generated test cases and handling edge cases correctly")

run_tests()
```

```
C:\Users\atur\OneDrive\Desktop\Ai_assistant>python -u "c:\Users\atur\OneDrive\Desktop\Ai_assis
Email validation function passing all AI-generated test cases and handling edge cases correctly
```

Explanation :

- The function checks whether the email contains required symbols like @ and . and ensures it does not start or end with special characters.
- It also verifies that there is only one @ symbol and handles invalid inputs safely.
- All assert test cases confirm the function works correctly.